

KVK Database Management System

Information brief for presentation and handover. This document explains what the system is, which records it stores, what changed, and what every project file is for.

Prepared on 03 October 2026. Database: kvk_db on this computer (localhost, port 5432). Passwords are kept in the local .env file and are not written in this PDF.

1. What this system actually is

This is a Windows desktop application for a Krishi Vigyan Kendra office. Staff use it to record farmer visits and KVK programmes, then search, check, analyse, and print those records. It is not a website. The window is built with PyQt5. The records live in one PostgreSQL database named kvk_db. Python reads and writes that database through SQLAlchemy.

One KVK office signs in. There are two working accounts already in the design: Admin and Staff. Admin can manage users and restore a backup. Staff can enter data in most modules. On Farm Testing, Front Line Demonstrations, and Extension Activities stay read-only for a staff login. No extra roles were added.

The day-to-day record is an activity. A farmer is stored once and linked to one or more activities. Visitor Farmers, On Farm Testing (OFT), Front Line Demonstrations (FLD), Training, Vocational Training, Extension Activities, and Other Extension Activities are all stored in the same activities table. The screen changes the fields. The database does not create a separate database for each module.

2. How a record is saved

Staff open a module, fill the form, and press Save. The program checks the mobile number (10 digits), then looks for an existing farmer with the same name, village, and mobile. If that farmer already exists, the new activity is attached to the same Farmer ID. If not, a new farmer row is created and given the next permanent ID, such as KVK-F-000011. The ID is never regenerated later.

Deleting a row removes that activity. If the farmer has no activity left, the farmer row is also removed from PostgreSQL. If the same farmer still has another visit, OFT, training, or extension record, the farmer stays.

3. Records in the database right now

These counts were read from the live kvk_db database while this PDF was created. They are the Maharashtra demonstration records plus normal office use. Refresh pgAdmin if an older view is still open.

Table	Rows now	What the row means
users	1	Sign-in account. Password is stored only as a PBKDF2-SHA256 hash.
departments	7	Agronomy, Horticulture, Plant Protection, Veterinary Science, Soil Science, Home Science, Agricultural Extension.
farmers	41	One person. Every row has a Farmer ID such as KVK-F-000048, plus name, village, and mobile.
activities	45	One programme row: visit, OFT, FLD, training, vocational training, or extension work.
reports	5	A note that a PDF or CSV report was generated, with the filters used.
audit_logs	66	Who did what, and when: login, save, update, delete, backup, import, settings.
app_settings	6	KVK name, address, backup folder, session timeout, and import status.
duplicate_reviews	0	Pairs of farmers that staff have marked Keep both. Empty until someone reviews a pair.

Activities by module

Module	Records
Visitor Farmers	10
On Farm Testing (OFT)	6
Front Line Demonstrations (FLD)	6
Training Programmes	7
Vocational Training Programmes	3
Extension Activities	8
Other Extension Activities	5
Total activities	45

4. What staff can do on each screen

Screen	What it is for
Sign in	Full-screen login. Wrong password is shown on the page. Five failures lock that username for five minutes.
Overview	Recently added records, recently updated records, and in-app alerts.
Visitor Farmers	Farmer visit: name, Farmer ID, village, mobile, district, tehsil, date, department.
On Farm Testing (OFT)	Trial title, crop, farmers, scientist, area, and season where the department uses it.
Front Line Demonstrations (FLD)	Demonstration record. Same style of form as OFT, with department-specific season rules.
Training Programmes	Training title, venue, clientele, farmers, dates, and scientist.
Vocational Training	The same training record, marked as vocational.
Extension Activities	Venue, location, purpose, and number of farmers. No separate farmer profile is required.
Other Extension Activities	Short extension item such as a news note or other outreach.
Analytics	Counts and charts: farmers, each module, participation, department, season, month, and village.
Reports	Filter, preview, PDF, CSV, and print. Columns follow the selected department and module.
Data Quality	Complete and incomplete farmers, missing mobile or village, bad dates, bad numbers, possible duplicates.
Duplicates	Possible duplicate farmers. Staff can review, keep both, open the record, or merge after confirmation.
Import	Excel or CSV: preview, map columns, validate, show duplicates, then confirm before anything is saved.
Calendar	Month view of training, OFT, FLD, extension, and vocational dates.
Village Analysis	Farmers, activities, OFT, FLD, and training participation for each village.
Year Comparison	Two financial years, April to March. Example: 2025-26 beside 2026-27.
Alerts	Incomplete records, duplicates, unusual values, overdue backup, and import problems. No SMS or email.
Settings	KVK name and address, backup folder, session timeout, new department, and change password.
Backup and Restore	PostgreSQL SQL backup. Shows last backup time. Restore asks for confirmation and checks the file.
User Management	Admin only. Create a user and change active status.
Search	Header box. Finds name, Farmer ID number, mobile, village, and programme titles.
Logout / Exit	Logout returns to sign-in. Exit closes the program after confirmation.

5. Farmer ID and search

Every farmer has one ID in the form KVK-F-000011. The number is the farmer's permanent database number, padded to six digits. Searching 11, 011, or 000011 finds that same farmer. Searching 48, 048, or 000048 finds KVK-F-000048. The full ID also works. The ID is shown on the visitor, OFT, and training tables and in reports.

6. Reports, analytics, and checks

Reports can be limited by date range, department, module, season, village, farmer, and activity type. The PDF shows the KVK name, title, filters, generated date, table, and page number. Season is included for Agronomy OFT and Horticulture FLD. Other department and module combinations leave season out, because those forms do not use it. CSV uses the same columns.

Analytics already had charts and a pivot table. The pivot is still the same screen. Values can be activity count, participant count, training count, OFT count, or FLD count. Rows and columns can be department, village, activity type, season, or year.

Anomaly detection looks at participant counts. When there is enough history it flags a value far above the normal range and asks staff to verify it. It does not delete or change the record. Data quality and duplicate review also leave the final decision with staff.

7. What changed

The original program was already a PyQt5 desktop system with login, seven activity modules, reports, backup, users, and an analytics pivot. The work below was added or completed on that same program. It was not rebuilt as a website.

Area	What changed
Sign-in and look	Full-screen sign-in, light green office theme, clearer tables, and a loading circle while the database, sign-in, search, lists, reports, analytics, pivot, and import are working.
Farmer ID	Permanent KVK-F-000001 style ID, shown on lists and reports, searchable by the number alone.
Delete	Deleting the last activity for a farmer now removes that farmer from PostgreSQL as well. Farmers who still have another activity stay.
Search	One search box for name, ID number, mobile, village, OFT, FLD, training, and extension titles. A result opens the record.
Import	Excel/CSV preview, column mapping, validation, duplicate list, and confirmation before save.
Quality and duplicates	Data quality counts, duplicate review, keep both, and a confirmed merge.
Office screens	Calendar, recent records, alerts, year comparison, village analysis, and settings.
Reports	More filters, department-wise columns, KVK header, page numbers, PDF, CSV, and print.
Analytics	Separate totals for visitor farmers, OFT, FLD, training, vocational training, and both extension modules. Existing charts and pivot kept.
Security	Change password, logout, 30-minute session timeout, failed-login tracking. Hashing stays PBKDF2-SHA256.
Backup and audit	Last backup time, backup-file check, restore warning. Audit now includes login, logout, import, settings, and password change.
Demo data	Maharashtra villages and programmes were entered for the presentation. Counts are in section 3.
Packaging	Windows file: dist\KVKSystem.exe, with .env kept beside it. Run from source with <code>py -3.12 main.py</code> .
GitHub	Source was pushed to github.com/Ravi8788/KVK . The .env password file and database backup files were not uploaded.

8. Why this technology is used

Technology	Why it is here
Python	The language of the whole program: screens, checks, and database work.
PyQt5	Windows desktop windows, forms, tables, and the calendar. Staff do not need a browser.
PostgreSQL	One reliable database for farmers, activities, users, settings, and audit history.
SQLAlchemy	Talks to PostgreSQL in Python without scattering raw connection code through the screens.

Technology	Why it is here
psycopg2	The driver that actually connects Python to PostgreSQL.
python-dotenv	Reads the database host, name, and password from the .env file next to the program.
Passlib PBKDF2-SHA256	Stores passwords as a hash. The original password is not kept in the database.
Pandas and openpyxl	Read Excel and CSV files during import, and check the rows before saving.
NumPy	Used when there is enough history to judge an unusual participant count.
ReportLab	Builds the PDF reports and this information document.
PyInstaller	Packages the program as KVKSystem.exe for a Windows PC.

Charts on the analytics screen are drawn inside PyQt5. Matplotlib and Seaborn are not required for those charts. The database connection path is: .env, then database/config.py, then database/session.py, then the service that saves or reads the record.

9. What each file is for

Start, database, and models

File	Contents and use
main.py	Starts the program, shows the loading circle, connects to PostgreSQL, creates missing tables, then opens sign-in.
run.bat	Small Windows launcher for the Python program.
.env	Local connection settings. This file stays on the PC and is not in GitHub.
requirements.txt	The Python packages to install, with versions.
KVKSystem.spec	PyInstaller instructions for building the exe.
database/config.py	Builds the PostgreSQL address from .env.
database/session.py	Opens and reuses the database connection.
database/init_db.py	Creates tables, adds new columns safely, fills Farmer IDs, and seeds the seven departments and settings.
database/schema.sql	A written snapshot of the tables. The running program updates the database through init_db.py.
models/base.py	The base class every table model uses.
models/entities.py	The table definitions: users, departments, farmers, activities, reports, audit logs, settings, and duplicate reviews.
models/__init__.py	Makes those models available to the rest of the program.

Screens

File	Contents and use
ui/login_window.py	Sign-in window and the handoff to the main window.
ui/dashboard_window.py	Top navigation, search box, logout, session timeout, and the page for each module.
ui/activity_form_widget.py	The data-entry forms and tables for all seven activity modules.
ui/oft_flow_widget.py	An older OFT-only form kept in the project. The dashboard uses activity_form_widget.py.
ui/reports_widget.py	Report filters, on-screen table, PDF, CSV, and print.
ui/smart_widgets.py	Overview, data quality, duplicates, import, calendar, village analysis, year comparison, alerts, settings, and the search window.
ui/backup_widget.py	Backup and restore buttons, last backup time, and the confirmation steps.
ui/user_management_widget.py	Admin screen for accounts.
ui/loading_overlay.py	The green loading circle.
ui/styles.py	Colours, spacing, buttons, and tables. Light background with green accents.

Services, reports, and analytics

File	Contents and use
controllers/auth_controller.py	Checks the password, locks repeated failures, creates the first admin if missing, and changes a password.
services/activity_service.py	Saves, updates, lists, searches, and deletes activities and farmers. Assigns the Farmer ID.
services/report_service.py	Applies report filters and chooses the columns for each department and module.
services/smart_service.py	Search, data quality, duplicates, anomalies, calendar, villages, year comparison, alerts, settings, and import checks.
services/backup_service.py	Runs pg_dump and restore, checks the SQL file, and aligns database ID sequences.
services/audit_service.py	Writes one audit row for an important action.
services/user_service.py	Creates users and updates whether an account is active.
reports/pdf_builder.py	Draws the activity PDF with the KVK heading and page numbers.
analytics/analytics_service.py	Counts, charts data, and the pivot query.
analytics/analytics_widget.py	The analytics and pivot screens, including the drawn charts.

Guides already in the folder

README.md is the short start guide. SETUP_AND_PACKAGING.md explains installation and the exe. The other Markdown and text files are deployment notes for running the same desktop program from another PC. They are instructions, not part of the running screens. info system.pdf is this briefing.

10. How to open it

From the project folder:

- cd C:\Users\ravir\Downloads\KVK-main\KVK-main
- py -3.12 main.py

Or start dist\KVKSystem.exe. The .env file must sit in the same folder as the exe. Sign in with the office account. The loading circle appears first, then the sign-in page, then the workspace.

This PDF is the current description of the Smart KVK desktop system: one Windows program, one PostgreSQL database, and the records listed above.